



UNIVERSIDADE DO MINHO
ESCOLA DE ENGENHARIA

RELATÓRIO

PROCESSAMENTO DE LINGUAGENS
COMPILADOR FORTH

Grupo 35:

A91672 LUÍS FERREIRA
A93258 BERNARDO LIMA

Professor:

José Carlos Ramalho

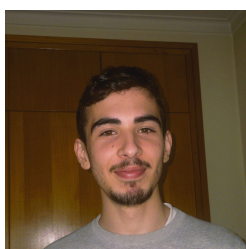




Tabela de conteúdos

1	Introdução	2
1.1	Apresentação do Problema	2
1.2	Objetivo	2
2	Análise e Especificação	3
2.1	Descrição informal do problema	3
2.2	Especificação dos Requisitos	3
3	Desenho da Solução	4
3.1	Analizador léxico	4
	Estados	4
	Tratamento de Tokens	5
3.2	Gramática	6
	Exp	6
	Strings	7
	Condicionais	9
	funStarted e functionBody	9
	Definição de funções	9
4	Testes Realizados e resultados	11
4.1	Alternativas, Decisões e Problemas de Implementação	11
4.2	Testes realizados e Resultados	11
5	Conclusão	13

1 Introdução

Neste trabalho, efetuado com o auxílio da linguagem de programação *Python* e do módulo *ply*, desenvolvemos um programa capaz de fazer a análise léxica e sintática da linguagem *Forth* e transformar o código *Forth* em código *Assembly*.

A análise sintática recebe como entrada o código que é identificado como uma sequência de *tokens*, através da análise léxica. Os *tokens* representam elementos básicos da linguagem, como palavras-chave, identificadores, operadores e símbolos especiais.

1.1 Apresentação do Problema

O desafio proposto neste projeto consiste na implementação de um compilador para a linguagem *Forth*, com o intuito de gerar código compatível com a máquina virtual desenvolvida no âmbito desta unidade curricular. Tanto a linguagem *Forth* quanto o código da máquina virtual foram explorados e discutidos nas aulas teóricas, proporcionando uma base de conhecimento sólida para o desenvolvimento deste compilador.

Entre os requisitos mínimos de implementação deste compilador estão o suporte a todas as expressões aritméticas, criação de funções, impressão de caracteres e strings, condicionais, ciclos e variáveis.

1.2 Objetivo

Este trabalho prático tem como objetivo ganhar experiência em engenharia de linguagens e programação generativa, aprimorando a nossa capacidade de elaborar gramáticas. Além disso, fortalece e melhora a capacidade de desenvolver processadores de linguagens utilizando o método da tradução dirigida pela sintaxe, a partir de uma gramática tradutora. O objetivo é desenvolver um compilador capaz de gerar código para um propósito específico. Para que seja possível atingi-lo, são utilizados geradores de compiladores baseados em gramáticas tradutoras, com destaque para o *Yacc*, do *PLY*, complementado pelo gerador de analisadores léxicos *Lex*, também do *PLY* do *Python*.

2 Análise e Especificação

2.1 Descrição informal do problema

O problema abordado neste trabalho consiste na implementação de um compilador para a linguagem Forth. A linguagem Forth é uma linguagem de programação de baixo nível e extensível, baseada em pilhas, que se destaca pela sua simplicidade e eficiência. O compilador deve ser capaz de converter código Forth para código de máquina compatível com a máquina virtual.

A principal dificuldade desse problema reside na complexidade da gramática da linguagem Forth, que é baseada em conceitos como *stacks*, operadores e estruturas de controlo. Além disso, é necessário implementar análise léxica e sintática para identificar corretamente os tokens e estruturas da linguagem.

2.2 Especificação dos Requisitos

Com base na descrição informal do problema, os requisitos mínimos para o compilador Forth são os seguintes:

- Implementar análise léxica e sintática para reconhecer corretamente os tokens e estruturas da linguagem Forth.
- Suportar todas as expressões aritméticas, incluindo adição, subtração, multiplicação, divisão e operações de módulo.
- Permitir a definição e chamada de funções.
- Possibilitar a impressão de strings.
- Implementar estruturas de controlo condicionais, como 'if', 'then' e 'else'.
- Garantir que o compilador gere código de máquina compatível com a máquina virtual.

Esses requisitos serão a base para o desenvolvimento e teste do compilador Forth ao longo deste trabalho.

3 Desenho da Solução

O nosso trabalho pode ser dividido em **2** partes:

- Construção do **analisador léxico**.
- Construção do **analisador sintático**.

3.1 Analisador léxico

O analisador léxico, é o responsável por 'capturar' os símbolos terminais(tokens) da nossa linguagem através de expressões regulares. Para a implementação do analisador léxico utilizamos o módulo 'Lex' do 'PLY/Python'.

Os tokens e respetivas expressões regulares da nossa linguagem são os seguintes:

```
NUMBER : '\d+'
ADD : '\+'
SUB : '\-'
MUL : '\*'
DIV : '/'
MOD : '%'
EQUALS : '='
DIFF : '<>'
GREQUAL : '>='
LESEQUAL : '<='
LESSER : '<'
GREATER : '>'
COMMENT : '\\\s.+\\n'
COMMENT2 : '\\(\s.+\\s\)'
FUNCTIONSTART : ':\s'
FUNCTIONEND : ';(?!.*;)'
FUNIN : '[^-\s\\n]+'
FUNOUT : '[^\s\\)\n]+'
LPAREN : '\('
RPAREN : '\s\)'
ARGSEP : '\s--\s'
WORD : '[A-Z0-9][A-Z0-9]+'
POPPRINT : '\.'
PRINTDELIM : '\.\"\\s[\"^]+\s\"'
EMIT : 'EMIT\b'
KEY : 'KEY\b'
SPACES : 'SPACES\b'
SPACE : 'SPACE\b'
CHAR : 'CHAR\b'
CR : 'CR\b'
IF : 'IF\b'
THEN : 'THEN\b'
ELSE : 'ELSE\b'
DO : 'DO\b'
LOOP : 'LOOP\b'
SWAP : 'SWAP\b'
```

Estados

Os estados definem diferentes modos de análise léxica no compilador Forth, adaptando o comportamento do analisador léxico de acordo com o contexto do código. Aqui estão os estados utilizados no compilador Forth:

- **functionDefState**: Este estado é utilizado para definir a estrutura de uma função Forth. Durante este estado, o analisador léxico está focado na captura dos parâmetros de entrada e saída da função. Ele é designado como inclusivo, o que significa que todas as funções que tiverem esse estado no nome serão adicionadas a este estado durante a análise léxica. Isso permite que o analisador léxico identifique corretamente a estrutura de definição de funções no código e capture os parâmetros corretamente.
- **funInpState**: Este estado é exclusivo e é ativado durante a declaração de input de funções. Quando o analisador léxico entra neste estado, apenas as funções com o nome do estado podem ser utilizadas. Isso garante que apenas tokens relacionados à entrada de funções sejam reconhecidos durante esta fase da análise léxica, facilitando a captura dos parâmetros de entrada da função.
- **funOutState**: Assim como o estado anterior, o **funOutState** é exclusivo e é utilizado durante a declaração de output de funções. Durante este estado, apenas as funções com o nome do estado podem ser usadas. Isso assegura que apenas tokens relacionados à saída de funções sejam reconhecidos durante esta fase da análise léxica, garantindo a captura precisa dos parâmetros de saída da função.
- **conditionalState**: Este estado é inclusivo e é ativado quando o analisador léxico encontra uma estrutura condicional no código, como um 'if'. Durante este estado, o analisador léxico está focado na análise da expressão condicional e de seus blocos de código associados. Designar este estado como inclusivo permite que todas as funções que tiverem esse estado no nome sejam adicionadas a este estado durante a análise léxica, facilitando a identificação correta dos tokens relacionados a estruturas condicionais no código.

Falar da transição de estados

Os estados no compilador Forth são fundamentais para garantir uma análise léxica precisa e eficiente, adaptando o comportamento do analisador léxico de acordo com o contexto do código.

Tratamento de Tokens

O tratamento dos tokens no analisador léxico do compilador Forth é crucial para identificar corretamente os elementos da linguagem e processá-los de acordo com as regras definidas. Existem diferentes tipos de tokens no Forth, cada um com suas características específicas e tratamentos necessários.

WORD

O token **WORD** representa palavras-chave e identificadores na linguagem Forth. Durante o tratamento desse token, é importante verificar se a palavra é uma palavra-chave reservada da linguagem ou um identificador definido pelo usuário.

NUMBER

O token **NUMBER** captura números inteiros na linguagem Forth. Durante o tratamento desse token, é necessário converter a sequência de caracteres em um valor numérico adequado, levando em consideração possíveis sinais, underscores entre os dígitos e representações em hexadecimal, octal e binário.

COMMENT

O token **COMMENT** identifica comentários no código Forth. Durante o tratamento desse token, o comentário pode ser ignorado pelo compilador, já que não afeta a execução do programa.

STRING

O token **STRING** representa strings na linguagem Forth. Durante o tratamento desse token, é necessário manipular as strings de acordo com suas características específicas, como delimitadores e escapes, garantindo que o compilador seja capaz de processá-las corretamente.

Além desses tokens principais, o compilador Forth pode definir outros tokens para operadores, estruturas de controle e símbolos especiais, cada um exigindo um tratamento específico durante a análise léxica para garantir a correta interpretação do código.

O tratamento adequado dos tokens no analisador léxico é fundamental para garantir que o compilador Forth seja capaz de reconhecer e processar corretamente o código, transformando-o em instruções compreensíveis para a máquina virtual.

3.2 Gramática

O analisador sintático é responsável por analisar a estrutura gramatical do código Forth e construir a árvore de análise sintática correspondente. Utilizamos o gerador de analisadores sintáticos 'Yacc' do 'PLY/Python' para implementar o analisador sintático. A gramática da linguagem Forth é definida através de regras de produção que especificam como os tokens devem ser combinados para formar instruções válidas.

Para auxiliar a conversão de Forth para linguagem da *VM*, foram implementadas as seguintes estruturas auxiliares, acopladas ao objeto responsável pela análise sintática, o *parser*:

- **enderecos**: Este dicionário é responsável por armazenar informações sobre funções definidas no input e tem duas componentes:
 - **'body'**: Armazena o corpo de funções definidas no ficheiro de input, de modo a ser possível reutilizar o código da *VM* gerado.
 - **'saldo'**: Contabiliza o estado da stack principal, de modo a verificar possíveis chamadas inválidas de funções.
- **func**: Armazena o nome da função a ser definida, de modo a ser possível adicionar funções já definidas dentro de outras.
- **funCount**: Counter que é responsável por fazer um cálculo sobre as modificações que uma determinada função ao ser definida provoca na stack principal.
- **i**: Counter que contabiliza o número de inteiros existentes na stack principal.
- **labels**: Counter que contabiliza o número de labels existentes no código gerado.
- **funDef**: Flag que indica se uma expressão está a ser deduzida dentro da definição de uma função.

Com esse objetivo, a gramática tem o símbolo não terminal **exps** como **axioma**. Deste símbolo, são derivadas duas produções:

```
1 def p_exps(p):
2     '''exps : exps exp'''
3     p[0] = p[1] + p[2]
```

```
1 def p_exps_empty(p):
2     '''exps : empty'''
3     p[0] = ""
```

A primeira, servindo para agrupar todas as expressões que constituem instruções válidas a ser introduzidas na máquina virtual *VM*. A segunda, para possibilitar a primeira no caso da primeira expressão obtida.

Exp

De forma a suportar o axioma, cabe ao símbolo não terminal **exp**, que deriva a grande maioria das expressões, maioritariamente, **funções**, quer as predefinidas, quer as definidas pelo utilizador, **expressões aritméticas**, **expressões lógicas**, **condicionais** e sobre **strings**.

```
1 def p_exp_word(p):
2     '''exp : WORD'''
3     p[0] = ""
4     if p[1] not in parser.enderecos and p[1] != parser.func:
5         print("Function '" + p[1] + "' not defined.")
6         raise SyntaxError
7
8     funInfo = parser.enderecos.get(p[1]) # Encontra a fun o no dicionario
9
10    if parser.funDef == False:
11        p[0] = funInfo['body'] # Adicionar corpo da fun o ao c digo
12        parser.i += funInfo['saldo'] # Adiciona o saldo da fun o ao contador
13    else:
14        p[0] = funInfo['body']
15        funAdef = parser.enderecos.get(parser.func)
16        funAdef['saldo'] += funInfo['saldo']
```

Quando uma função é encontrada, existe primeiramente uma verificação sobre a existência de uma definição da mesma (que iremos abordar mais tarde). Após isso, o corpo da função é copiado do `parser.enderecos`, de modo a ser inserido no código gerado, assim como é adicionado ao `parser.i` o 'saldo' da função, de modo a registrar quantos inteiros existem na stack com a chamada da função.

De modo a lidar com expressões aritméticas, estão associadas as seguintes derivações da *exp*:

```

1  def p_exp_aritOnly(p):
2      '''exp : ADD
3              | SUB
4              | MUL
5              | DIV
6              | MOD'''
7
8      if parser.funDef == False:
9          if parser.i < 2:
10             print("Arithmetic Error: Not enough values on stack!")
11             parser.i -= 1
12         else:
13             parser.funCount -=1
14
15         if p[1] == '+':
16             p[0] = 'ADD\n'
17         elif p[1] == '-':
18             p[0] = 'SUB\n'
19         elif p[1] == '/':
20             p[0] = 'DIV\n'
21         elif p[1] == '*':
22             p[0] = 'MUL\n'
23         elif p[1] == '%':
24             p[0] = 'MOD\n'
25
26 def p_exp_relOpOnly(p):
27     '''exp : LESSER
28             | GREATER
29             | DIFF
30             | GREQUAL
31             | LESEQUAL
32             | EQUALS'''
33
34     if parser.funDef == False:
35         if parser.i < 2:
36             print("Logical Error: Not enough values on stack!")
37             raise SyntaxError
38         parser.i -= 1
39     else:
40         parser.funCount -=1
41
42     if p[1] == '<':
43         p[0] = 'INF\n'
44     elif p[1] == '>':
45         p[0] = 'SUP\n'
46     elif p[1] == '<>':
47         p[0] = 'NOT\n'
48     elif p[1] == '<=':
49         p[0] = 'INFEQ\n'
50     elif p[1] == '>=':
51         p[0] = 'SUPEQ\n'
52     elif p[1] == '=':
53         p[0] = 'EQUAL\n'

```

Ambas possuem o mesmo raciocínio lógico. Primeiramente no caso de não estarem a ser incluídas numa definição de função, verifica-se se existem valores na stack principal para ser possível realizarem essa operação. Caso afirmativo, é contabilizada a diferença na stack principal através da *funCount*. Caso contrário, é subtraída ao contador global de inteiros na stack, o *parser.i*. E finalmente é adicionado o código ao *output*.

Strings

Também derivadas da *exp*, servem para lidar com todas as funções propostas da linguagem Forth que lidam com *strings*:

```

1  def p_exp_charOnly(p):
2      '''exp : CHAR WORD'''
3
4      if len(p[2]) > 1:
5          print("SyntaxError - CHAR applied to more than one character.")
6          raise SyntaxError
7
8      p[0] = 'pushs ' + p[2] + '\n'
9      p[0] += 'CHRCODE\n'
10
11     if parser.funDef == False:
12         parser.i += 1
13     else:
14         parser.funCount +=1
15
16
17 def p_exp_printOnly(p):
18     '''exp : PRINTDELIM
19         | CR
20         | SPACE
21         | KEY'''
22     if p[1] == 'CR':
23         p[0] = 'writeln\n'
24     elif p[1] == 'SPACE':
25         p[0] = 'pushs " " \n'
26     elif p[1] == 'KEY':
27         p[0] = 'read\n'
28         p[0] += 'chrcode\n'
29         if parser.funDef == False:
30             parser.i += 1
31         else:
32             parser.funCount +=1
33     else: # PRINTDELIM
34         p[0] = 'pushs ' + p[1] + '\n'
35         p[0] += 'writes \n'
36
37
38 def p_exp_prints(p):
39     '''exp : SPACES
40         | EMIT'''
41
42     strt = '1' + str(parser.labels)
43     parser.labels +=1
44     fin = '1' + str(parser.labels)
45     parser.labels +=1
46     if parser.funDef == False:
47         if parser.i <= 0:
48             print('Syntax Error on SPACES or EMIT function. Not enough arguments.\n')
49             raise SyntaxError
50             parser.i -= 1
51         else:
52             parser.funCount -=1
53
54     if p[1] == 'SPACES':
55         #loop core 1
56         p[0] += strt + ':\n'
57         p[0] += 'dup 1\npushi 0\nsup\njz ' + fin + '\n'
58         #print espa o
59         p[0] += 'pushs " "\nwrites\n'
60         #loop core 2
61         p[0] += 'pushi 1\nsub\njump ' + strt + '\n'
62         p[0] += fin + ':\n' + 'pop 1\n'
63     else: # EMIT
64         p[0] = 'writechr\n'

```

Destas produções, é de destacar dois casos:

Primeiramente, as funções que não requerem elementos da stack principal, referentes à produção **print-Only**, que possuem um comportamento simples, aonde é apenas adicionado o código correspondente, salvo a exceção da KEY que possui um comportamento semelhante às ainda não referidas. Com foco nas produções que necessitam de elementos da stack principal, estas têm um comportamento semelhante às expressões aritméticas e lógicas.

Condicionais

Para a utilização de condicionais, também derivadas de **exp**, utilizam também o símbolo não terminal **exps**, de forma a não restringir o tamanho das ramificações lógicas possíveis.

```

1  def p_exp_ifThen(p):
2  '''exp : IF exps THEN'''
3  if parser.funDef == False:
4      if parser.i < 1:
5          print("Conditional Error: Not enough values on stack!")
6          raise SyntaxError
7      parser.i -= 1
8  else:
9      parser.funCount -= 1
10
11  false = '1' + str(parser.labels)
12  p[0] = 'JZ ' + false + '\n' # Salto caso seja falso
13  p[0] += p[2] # Condi o caso verdadeiro
14  p[0] += false + ':\n' # Salto (falso)
15  parser.labels+=1
16
17 def p_exp_ifElseThenOnly(p):
18 '''exp : IF exps ELSE exps THEN'''
19 if parser.funDef == False:
20     if parser.i < 1:
21         print("Conditional Error: Not enough values on stack!")
22         raise SyntaxError
23     parser.i -= 1
24 else:
25     parser.funCount -= 1
26
27 fi = '1' + str(parser.labels)
28 parser.labels+=1
29 els = '1' + str(parser.labels)
30 parser.labels+=1
31 fim = '1' + str(parser.labels)
32 p[0] = 'JZ ' + els + '\n' # Salto caso seja falso
33 p[0] += p[2] # V
34 p[0] += 'jump ' + fim + '\n' # Salto para o fim
35 p[0] += els + ':\n' # Salto F
36 p[0] += p[4] # F
37 p[0] += fim + ':\n' # fim
38
39 parser.labels+=1

```

Com destaque nestas produções, as **labels** e os *jumps*, que serão cruciais para possibilitar a navegação lógica, garantindo que não existem confusões nas mesmas. Garantindo que uma condicional não interfere com outra.

funStarted e functionBody

As produções derivadas destes símbolos não terminais surgem com a necessidade de ativar flags para detecção caso a leitura seja dentro da definição de funções.

Definição de funções

```

1  def p_funStarted_WORD(p):
2  '''funStarted : FUNCTIONSTART WORD'''
3  p[0] = ""
4  if p[2] in parser.enderecos:
5      print('Function ' + p[2] + ' already exists.')
6      raise SyntaxError
7  parser.funDef = True
8  parser.func = p[2]
9  data = ({p[2]: {'body': "", 'saldo': 0}})
10  parser.enderecos.update(data)
11
12
13 def p_functionBody(p):
14 '''functionBody : exps FUNCTIONEND'''

```

```
15 p[0] = ""
16 nome = parser.func
17 bodyFun = p[1]
18
19 parser.enderecos[nome]['body'] = bodyFun
20 parser.enderecos[nome]['saldo'] = parser.funCount
21
22
23 #Reiniciar Variaveis globais
24 parser.funCount = 0
25 parser.func = ""
26 parser.funDef = False
27
28
29 def p_exp_funDefined(p):
30     '''exp : funStarted functionBody'''
31     p[0] = ""
```

Assim, a produção *funStarted* fica responsável pela ativação da flag global anteriormente referida. Complementado esta produção, a *functionBody*, de modo a reiniciar parte da estrutura auxiliar global referente à definição de funções. Finalmente a produção *funDefined*, que é derivada da *exp*, mas é aqui inserida pois representa uma boa definição de funções.

4 Testes Realizados e resultados

4.1 Alternativas, Decisões e Problemas de Implementação

A implementação de um dicionário global, onde são armazenados os corpos das funções, impossibilita a chamada aninhada de funções dentro de outras funções que usem *labels*, o que limita bastante o desempenho do programa. Este problema poderia ter sido resolvido através da utilização de *frame pointers* e *global pointers* fornecidos na máquina virtual. Contudo, devido a falhas do nosso conhecimento da linguagem da máquina, não nos foi possível implementar essa opção.

4.2 Testes realizados e Resultados

Input:

3 +

Output:

Arithmetic Error: Not enough values on stack!

Input:

3 5 + 8 > IF ." Valor é superior " ELSE ." Valor é igual ou inferior." THEN

Output:

```
START
pushi 3
pushi 5
ADD
pushi 8
SUP
JZ 12
pushs "Valor é superior"
writes
jump 13
12:
pushs "Valor é igual ou inferior"
writes
13:
STOP
```

Input:

2 3 + .
2 3 + 10 + .

Output:

```
START
pushi 2
pushi 3
ADD
writei
pushi 2
pushi 3
ADD
pushi 10
ADD
writei
STOP
```

Input:

```
: TESTKEY ( -- )
." Hit a key: " KEY CR
." That = " . CR;
```



TESTKEY

Output:

```
START
pushs "Hit a key:"
writes
read
chrcode
writeln
pushs "That ="
writes
writei
writeln
STOP
```

Input:

```
: AVERAGE ( a b -- avg ) + 2/ ;
10 20 AVERAGE .
```

Output:

```
START
pushi 10
pushi 20
ADD
pushi 2
DIV
writei
STOP
```

5 Conclusão

Durante o desenvolvimento deste trabalho prático, enfrentamos diversos desafios e adquirimos uma compreensão mais profunda sobre as linguagens de programação e compiladores. Ao longo do processo, aplicamos os conhecimentos adquiridos na aula para criar uma gramática e implementar um compilador capaz de transformar código Forth em código de máquina compatível com a máquina virtual.

No entanto, mesmo com esforço e dedicação, encontramos algumas dificuldades significativas. Uma das principais dificuldades foi a compreensão e implementação do código da máquina virtual, que serve como plataforma de execução para o código gerado pelo nosso compilador. Demorou-nos bastante tempo para conseguirmos começar a compreender como o código da máquina funcionava e como traduzir para esta linguagem.

Outro desafio importante foi a implementação do tratamento de funções aninhadas, variáveis e ciclos no compilador Forth. Devido à complexidade desses recursos não conseguimos implementar esses aspectos na versão final do compilador.

Apesar das dificuldades encontradas, consideramos que alcançamos os objetivos principais do trabalho, ganhando prática e experiência. O processo de desenvolvimento proporcionou-nos uma compreensão mais sólida dos conceitos teóricos abordados em sala de aula e preparou-nos para enfrentar desafios semelhantes no futuro.